一次 FFN 矩阵乘法的多 GPU 划分

用 8 张 GPU 沿公共维度拆分计算，并以逐元素求和恢复完整输出

核心结论：每张 GPU 保存 1/8 的权重并计算一个部分乘积；8 份部分结果形状相同，通过逐元素求和得到完整输出。并行执行缩短了计算时间，总乘法次数保持不变。

1 计算任务与系统配置

Transformer 前馈网络（FFN）通过矩阵乘法变换 token 特征。本方案处理 128 个 token，每个 token 包含 8192 个输入特征，投影后生成 4096 个输出特征：

$$Y = XW \quad X \in R^{128\times 8192},\ W \in R^{8192\times 4096},\ Y \in R^{128\times 4096}$$

其中，X 是输入，W 是权重，Y 是输出，一行 X 对应一个 token。模型加载阶段，8 份权重块分别驻留在对应 GPU。GPU 1 分发输入块、汇总部分结果并输出 Y；一次推理依次经历输入分发、并行计算和结果合并。

2 怎么分：沿输入特征维度切成 8 份

将 X 按列切分，将 W 按对应行切分：$X = [X_1\ X_2\ \dots\ X_8]$，$W = [W_1; W_2; \dots; W_8]$。每个 X_i 为 128×1024，每个 W_i 为 1024×4096。每张卡独立计算 $P_i = X_iW_i$，局部计算期间无需交换数据。

GPU	X 的列 / W 的行	局部计算	局部结果
1	1–1024	$P_1 = X_1W_1$	128×4096
2	1025–2048	$P_2 = X_2W_2$	128×4096
3	2049–3072	$P_3 = X_3W_3$	128×4096
4	3073–4096	$P_4 = X_4W_4$	128×4096
5	4097–5120	$P_5 = X_5W_5$	128×4096
6	5121–6144	$P_6 = X_6W_6$	128×4096
7	6145–7168	$P_7 = X_7W_7$	128×4096
8	7169–8192	$P_8 = X_8W_8$	128×4096

为什么能并行？ 各卡读取的输入特征区间互不重叠，权重块也已就位；P_1 到 P_8 之间没有前后依赖。均匀划分使每张卡承担相同的乘法工作量。代价是每张卡都会产生一份完整形状的部分结果。

3 Divide → Conquer → Combine

DIVIDE	CONQUER	COMBINE
沿 8192 个输入特征连续二分，直到得到 8 个各含 1024 个特征的子任务。GPU 1 保留 X_1，其余输入块直接发送到对应 GPU。	第 i 张 GPU 计算 $P_i = X_iW_i$。原先每个输出元素累加 8192 项，现在每卡只累加自己负责的 1024 项。	GPU 2–8 将 P_i 发回 GPU 1，GPU 1 对同一输出位置逐元素累加，得到 $Y = P_1 + \dots + P_8$。

4 为什么要相加，而不是拼接

矩阵乘法的每个输出元素本来就是沿公共维度 k 求和。8 张卡覆盖的 k 区间互不重叠，合起来恰好覆盖 1 到 8192：

$$Y[a,b] = \Sigma_{k=1\dots8192}\, X[a,k]W[k,b] = \Sigma_{i=1\dots8}\,(\,\Sigma_{k=1024(i-1)+1\dots1024i}\, X[a,k]W[k,b]\,)$$

因此 $P_i[a,b]$ 都对应输出 Y 中同一个位置，只是包含的乘积项不同。8 份部分结果必须逐元素相加；直接拼接会改变输出的形状和含义。

5 数据流与通信位置

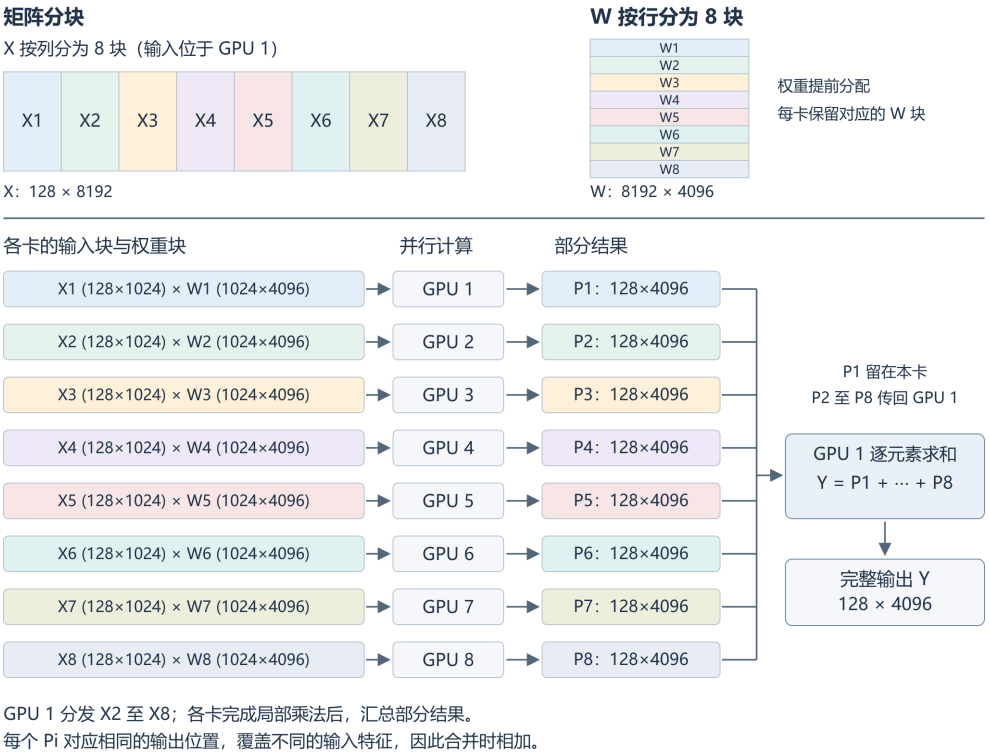


图 1 矩阵分块、8 张 GPU 的局部计算与集中汇总流程

数据交换：计算开始时，GPU 1 分发 X_2–X_8；局部乘法完成后，GPU 2–8 回传 P_2–P_8。W_i 常驻对应 GPU，局部计算阶段无需交换权重或中间数据。GPU 1 完成集中规约并输出 Y。

6 GPU 越多一定越快吗?

在相同的 X、W 尺寸下，GPU 数量从 1 增加到 16。各卡承担的乘法份额随之减小，每份部分结果始终为 128×4096：

GPU 数量	每卡输入特征	每卡乘法工作量	GPU 1 接收结果
1	8192	全部	0 份
2	4096	1/2	1 份
4	2048	1/4	3 份
8	1024	1/8	7 份
16	512	1/16	15 份

为什么一开始会加速

局部矩阵乘法可以同时执行。理想情况下，2 张卡把主要计算时间降至约 1/2，4 张卡降至约 1/4。总乘法次数保持不变，节省的是完成同一批计算所需的时间。

① 通信与合并增长
集中汇总时，GPU 1 接收的 128×4096 结果从 8 卡的 7 份增至 16 卡的 15 份，还要逐元素求和。输入分发同样占用链路。

② 子任务过小
矩阵变窄后，GPU 计算单元利用率下降；内核启动、调度等固定开销不会随工作量同比减少，小批量 token 时尤其明显。

③ 同步与慢卡效应
完整 Y 必须等待全部 P_i。一张卡计算偏慢或传输延迟，其他卡完成后仍需等待；卡数越多，尾部延迟越容易影响整体时间。

什么时候会更慢

总耗时由输入分发、最慢卡的局部计算、结果传输和求和共同构成。从 8 卡增至 16 卡，局部计算再节省单卡时间的 1/16；新增的通信、规约和等待一旦超过这部分收益，16 卡就会比 8 卡更慢。树形规约可以缓解集中瓶颈，通信和同步成本仍然存在。

7 与经典分治的联系

本方案与经典分治一样，把同一问题拆成较小的同类子问题，分别求解后再合并。分治描述算法结构，多 GPU 描述硬件分工；多卡并行缩短等待时间，不减少乘法总量，同时增加通信与同步。把互不相关的请求分给不同 GPU，没有拆解同一问题，也不需要合成答案，不属于分治。

参考资料

NVIDIA Megatron Core, RowParallelLinear：输入与权重沿公共维度分块，局部结果通过规约合并。
https://docs.nvidia.com/megatron-core/developer-guide/latest/apidocs/core/core.tensor_parallel.layers.html